

GENERATIVE AI

Hands on Experience



A clear, step-by-step journey into Generative AI



Disclaimer

EVERYONE LEARNS UNIQUELY.

A structured, hands-on journey into Generative AI - from core concepts to building real-world GenAI applications.

TABLE OF CONTENTS

01

INTRODUCTION TO GENERATIVE AI

What Is Generative AI?

Difference Between AI, ML, DL, And GenAI

04-07

02

CORE CONCEPTS OF GENERATIVE AI

Training Data & Learning Patterns

Probability, Tokens, And Embeddings

Latent Space And Representation Learning

08-12

03

GENERATIVE AI MODELS

Large Language Models (LLMs)

Diffusion Models

GANs (Generative Adversarial Networks)

Multimodal Models

13-19

04

HOW GENERATIVE AI WORKS

Model Training Process

Inference And Prompting

Zero-Shot, One-Shot, And Few-Shot Learning

20-27

05

PROMPT ENGINEERING

What Is Prompt Engineering?

Prompt Types & Structures

28-30

06

ARCHITECTURE OF GENAI SYSTEMS

----- 31-37

Model Layer

Infrastructure & Deployment

APIs & Integration

Security Considerations

07

FUTURE OF GENERATIVE AI

----- 38-42

Trends And Innovations

GenAI In Enterprises

Human + AI Collaboration

1. Introduction to Generative AI

1.1 What Is Generative AI?

Generative AI (GenAI) is a category of Artificial Intelligence that focuses on **creating new content** rather than only analyzing data or making predictions. The generated content can closely resemble **human-created outputs**.

Generative AI is capable of creating:

- Text (articles, emails, chat responses, summaries)
- Images (artwork, illustrations, designs)
- Videos and animations

Unlike traditional AI systems that provide fixed or predefined outputs, Generative AI produces **original and dynamic results** based on user input.

How Generative AI Works

Learns patterns, structure, and relationships from large datasets

- Understands context from the input prompt
- Generates new content that follows learned patterns
- Adapts responses based on the type and detail of input

Example:

- **ChatGPT** generating articles, answering queries, and writing code
- **DALL·E / Midjourney** creating images from text descriptions
- **GitHub Copilot** assisting developers by generating code in real time

These technologies represent different layers of capability within the AI ecosystem, each increasingly specialized.

1. Artificial Intelligence (AI)

Artificial Intelligence is the **broadest concept**, referring to machines that can perform tasks requiring human-like intelligence.

AI systems are typically used to:

- Make decisions
- Solve problems
- Follow logical rules

Example:

- Navigation systems suggesting optimal routes
- Chess-playing programs
- Rule-based customer support systems

2. Machine Learning (ML)

Machine Learning is a **subset of AI** where systems learn from data instead of being explicitly programmed.

| Key Characteristics Of ML:

- Learns from historical data
- Improves performance over time
- Identifies patterns automatically

Example:

- Recommendation systems (Netflix, Amazon)
- Spam detection in emails
- Fraud detection systems

3. Deep Learning (DL)

Deep Learning is a **subset of Machine Learning** that uses multi-layered neural networks inspired by the human brain.

Deep Learning excels at:

- Image recognition
- Speech processing
- Natural Language Understanding

Example:

- Face recognition in smartphones
- Voice assistants like Alexa or Siri
- Medical image analysis systems

4. Generative AI (GenAI)

Generative AI is a **specialized branch of Deep Learning** that focuses on **generating new content** instead of predicting or classifying data.

Generative AI systems can:

- Generate human-like text and conversations
- Create images from text prompts
- Generate source code and documentation
- Summarize or rewrite large amounts of data

Example:

- Chatbots generating human-like responses
- AI tools creating images from written descriptions
- Code-generation tools assisting software development

2. Core Concepts of Generative AI

To understand how Generative AI creates text, images, code, or other content, it is important to first understand the **core concepts** that power these systems. These concepts explain how models learn, process information, and generate outputs that appear intelligent and human-like.

2.1 Training Data & Learning Patterns

Generative AI models learn by analyzing **large amounts of training data**. This data acts as the foundation that teaches the model how language, images, or code are structured.

(a) Training data can include:

- Books, articles, and websites
- Conversations and question-answer data
- Images, videos, and audio
- Source code and documentation

(b) During training, the model does not memorize answers, but learns:

- How words relate to one another
- How sentences are structured
- How concepts are connecte

(c) The model identifies:

- Repeated patterns
- Contextual relationships
- Common formats and styles

Why This Is Important

- High-quality and diverse data results in **better accuracy**
- Poor or biased data can lead to **incorrect or biased outputs**
- The model's intelligence is limited by what it has learned from its data

Generative AI becomes more effective as it learns **patterns across vast datasets**, enabling it to generate meaningful new content rather than copying existing data.

2.2

Probability, Tokens, And Embeddings

Generative AI systems rely on **mathematical representations** to understand and generate content. Three key concepts make this possible.

1. Tokens

Tokens are the basic units that a model works with instead of full words or sentences.

(a) A token can be:

- A word
- Part of a word
- A symbol or punctuation mark

- Before processing input, text is broken down into tokens
- Models generate output **token by token**

Why Tokens Matter

- Longer responses use more tokens
- Token count affects speed, cost, and performance
- Tokenization allows models to work across multiple languages and formats

2. Probability

Generative AI does not choose fixed answers. Instead, it predicts the **most likely next token** based on probability.

(a) Each generated word is selected based on:

- Context
- Learned patterns
- Statistical likelihood

(b) This is why the same prompt can produce slightly different responses

(c) Parameters like temperature control:

- Creativity (higher randomness)
- Accuracy (lower randomness)

Why Probability Matters

- Enables flexibility in responses
- Allows creativity instead of rigid answers
- Powers natural and human-like text generation

3. Embeddings

Embeddings are numerical representations of meaning.

- Words, sentences, or documents are converted into vectors (numbers)
- Similar meanings result in embeddings that are closer together
- Embeddings help the model understand **context**, not just keywords

Uses of Embeddings

- Semantic search
- Content similarity
- Recommendation systems
- Question-answer matching

Embeddings allow AI systems to understand what users mean, not just what they type.

Latent space is a **hidden mathematical space** where learned knowledge is stored.

Latent Space

- Concepts are stored as abstract representations
- Similar concepts are located closer together
- This enables smooth transitions between ideas

Representation Learning

(a) The model learns how to represent:

- Language meaning
- Visual features
- Logical relationships

(b) Instead of hard-coded rules, the model develops internal understanding

3. Generative AI Models

Generative AI models are the **engines** that power content creation in modern AI systems. Each model type is optimized for a specific class of problems such as **language, images, speech, or combined modalities**.

Understanding these models helps you:

- Choose the right GenAI architecture
- Build scalable AI systems
- Explain AI behavior in production and interviews
- Evaluate strengths, limitations, and costs

3.1 Large Language Models (LLMs)

Large Language Models focus on **understanding and generating human language** at scale.

Core Idea

LLMs learn how language works by reading enormous amounts of text and learning:

- Meaning of words in context
- Grammar and structure
- Relationships between sentences
- Logical and semantic patterns

They do not store facts like a database they **learn probabilities of language**.

How LLMs Are Trained

- Input text is split into **tokens**
- Tokens are converted into numbers
- The model learns to predict the **next token**
- This process is repeated billions of times
- Transformers enable parallel learning and long-context understanding

| KEY CAPABILITIES:

- Text generation with context awareness
- Sentence completion and rewriting
- Question answering
- Code generation and debugging
- Logical reasoning and summarization

Example:

- ChatGPT for conversational AI
- GitHub Copilot for auto-completing code

3.2

Diffusion Models

Diffusion models are **probabilistic generative models** optimized for structured data like images and audio.

The Diffusion Process

Diffusion models learn:

$$P(x_0 \mid x_T)$$

Where:

- x_0 = clean data
- x_t = data after noise at time t

Two Phases

Forward Process

- Gradually adds noise
- Information is destroyed

Reverse Process (Learning Phase)

- Model learns to remove noise
- Noise $\rightarrow$ structure

Why Diffusion Models Work So Well

- Stable optimization
- Gradual refinement
- High detail preservation

Example:

- Image generation APIs
- Design tools
- Video synthesis
- Medical image reconstruction

3.3

GANs (Generative Adversarial Networks)

GANs model data using **game theory**.

Min-Max Optimization

GANs optimize:

$$\min_G \max_D V(D, G)$$

- Generator tries to minimize detection
- Discriminator tries to maximize detection

This is a **two-player zero-sum game**

Why GANs Are Hard to Train

- Non-convergent dynamics
- Mode collapse
- Gradient instability

When GANs Still Win

- Real-time image generation
- Low-latency environments
- Edge devices

Example:

- Anonymization systems
- Face synthesis
- Super-resolution

3.4

Multimodal Models

What does “Multimodal” mean?

A **modality** is a type of data.

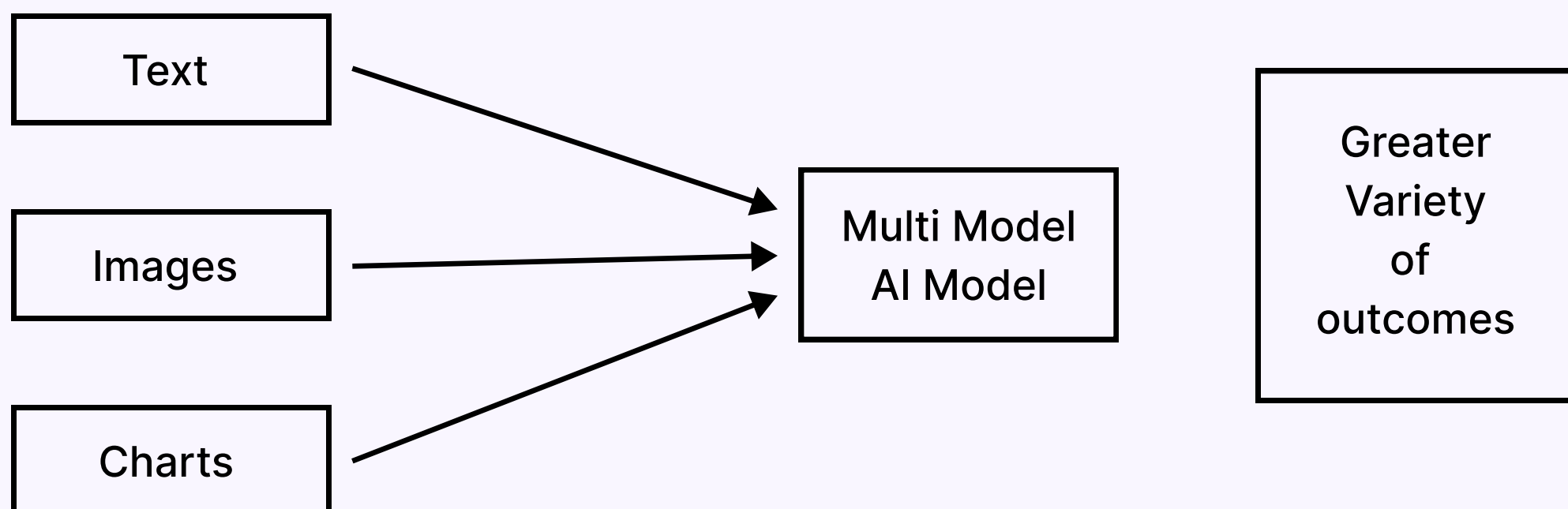
Example:

- Text
- Images
- Audio
- Video

Single Model



Multiple AI Model



What is a “Shared Latent Representation”?

Think of it like this:

- The model converts **text, images, audio** into **numbers (vectors)**
- Instead of keeping them separate, it puts them into **one shared “meaning space”**

This shared space is called the **latent space**.

Example:

Imagine you show a model:

- Image: a golden retriever playing with a ball
- Text: “A dog playing with a ball”

Inside a Multimodal Model:

1. Image encoder → converts image into numbers
2. Text encoder → converts text into numbers
3. Both are mapped **close together** in the same vector space

4. How Generative AI Works

Generative AI works by **learning patterns and probability distributions from large-scale data** and then **sampling from those distributions to produce new outputs** such as text, images, code, audio, or video.

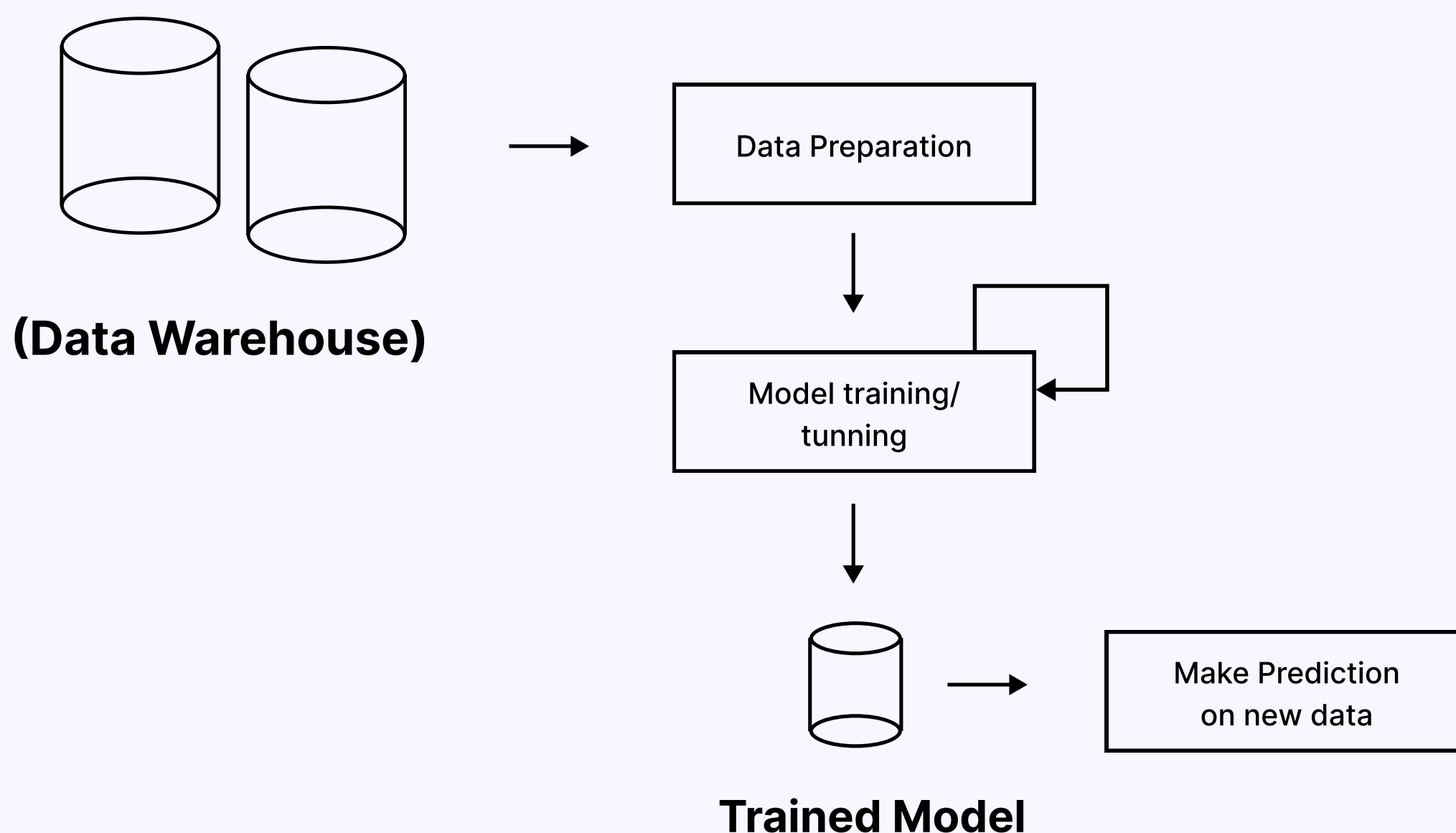
Unlike traditional software, GenAI systems **do not follow explicit rules**.

Instead, they operate on:

- Statistical learning
- Probabilistic reasoning
- High-dimensional vector representations

At a high level, GenAI works in **two major stages**:

1. **Model Training** (learning knowledge)
2. **Inference** (using that knowledge to generate output)



Model training is the most expensive and critical phase of Generative AI.

Step 1: Data Collection & Preprocessing

Models are trained on massive datasets:

- Text (books, articles, code repositories)
- Images (labeled and unlabeled)
- Audio and video snippets
- Multimodal pairs (image + text)

Before training:

- Text is cleaned, tokenized, and normalized
- Images are resized and augmented
- Noise and redundancies are reduced

Step 2: Tokenization & Numerical Encoding

Raw data cannot be processed directly.

Example:

- Words → tokens
- Tokens → numeric IDs
- Images → pixel tensors
- Audio → spectrogram arrays

Everything is converted into **numbers**, allowing:

- Matrix operations
- Gradient computation
- Efficient GPU processing

Step 3: Objective Function (What the Model Learns)

Most Generative AI models are trained using:

$$\text{Loss} = -\log P(\text{actual_output} \mid \text{context})$$

This means:

- The model predicts an output
- Loss increases if the prediction is wrong
- Gradients guide the model toward better predictions

LLMs specifically minimize **next-token prediction loss**.

Step 4: Backpropagation & Optimization

Training uses:

- Backpropagation
- Gradient descent variants (Adam, AdamW)
- Billions to trillions of parameter updates

Over time:

- Internal representations improve
- Language patterns stabilize
- Meaning emerges from structure

4.2

Inference And Prompting

Inference is the **runtime phase** where trained models generate outputs.

Inference = Controlled Sampling

Once trained, the model:

- Receives an input prompt
- Converts it to vectors
- Predicts probability distributions
- Samples an output token-by-token

This process continues until:

- A stop token is reached
- A length limit is hit

Prompting: The Control Mechanism

Prompts act as **initial conditions** for probability generation.

A prompt defines:

- Task structure
- Output format
- Reasoning style
- Level of detail

4.3 Zero-Shot, One-Shot, And Few-Shot Learning

Before understanding zero, one, and few-shot learning, it's important to understand how models "learn" during inference.

LLMs do not learn by updating weights during inference. Instead, they:

- Analyze the prompt
- Detect patterns from instructions or examples
- Generate responses that best match those patterns

1. Zero-shot Learning

In zero-shot learning, the model is asked to perform a task without seeing any prior

Example:

- Knowledge learned during pre-training
- Understanding of language semantics
- Instruction-following capability

Because LLMs are trained on diverse datasets containing instructions, discussions, and task descriptions, they develop an internal understanding of how common tasks work (classification, translation, summarization, reasoning, etc.).

The model predicts the most probable output sequence that satisfies the instruction.

Working

- You give the model a clear instruction
- The model interprets the intent
- It generates an answer using its pre-learned knowledge

Example:

Classify the sentiment of the sentence:
"I enjoyed the workshop."

Model Output

Positive

2. One-shot Learning

In one-shot learning, the model is shown a single example that demonstrates the expected input-output format. This helps the model:

- Infer the task structure
- Align with the expected response style
- Reduce ambiguity in interpretation

LLMs are exceptionally good at pattern recognition, even from a single example. The example acts as a temporary reference that the model imitates when generating the next response.

Working

You show one solved example

- The model understands the pattern
- It applies the same logic to the new input

Example:

Sentence: "The app works flawlessly"

Sentiment: Positive

Sentence: "The app crashes frequently"

Sentiment: Negative

3. Few-shot Learning

In few-shot learning, multiple examples are provided before the task is asked. This allows the model to:

- Establish stronger patterns
- Understand edge cases
- Maintain consistent reasoning and style

Each additional example narrows down uncertainty and helps the model form a more accurate internal representation of the task.

Few-shot prompts are especially valuable in production systems where consistency is critical.

Working

- You give several examples
- The model detects repeated patterns
- It confidently applies those rules to new input

5. Prompt Engineering

Prompt Engineering is the skill of communicating effectively with Generative AI models (like ChatGPT, GPT-4, Claude, Gemini) so that they produce accurate, useful, and reliable outputs.

Think of it as writing instructions for AI in a way the model understands best.

5.1 What Is Prompt Engineering?

Prompt Engineering is the process of designing, refining, and structuring input prompts to guide an AI model toward a desired output.

A prompt is not just a question it can include:

- Instructions
- Context
- Constraints
- Examples
- Formatting rules
- Tone and role definitions

The quality of the prompt directly impacts:

- Accuracy of responses
- Depth of reasoning
- Consistency of output
- Reduction of hallucinations

| KEY IDEA:

AI doesn't "understand" intent like humans. It predicts outputs based on patterns so how you ask matters more than what you ask.

Why Prompt Engineering Is Important

- Saves time by reducing follow-up questions
- Improves response quality without retraining models
- Acts as a lightweight alternative to fine-tuning
- Critical for production AI systems and tools
- Widely tested in AI interviews and real-world use cases

5.2 Prompt Types & Structures

Different prompts serve different goals. Below are the most commonly used prompt types.

1. Instruction Prompts → Direct instructions telling the model what to do.

Example: Summarize the following article in 5 bullet points.

2. Role-Based Prompts → You are a senior backend engineer. Explain database indexing to a junior developer.

Example: You are a senior backend engineer. Explain database indexing to a junior developer.

3. Contextual Prompts

You give background information before asking the task.

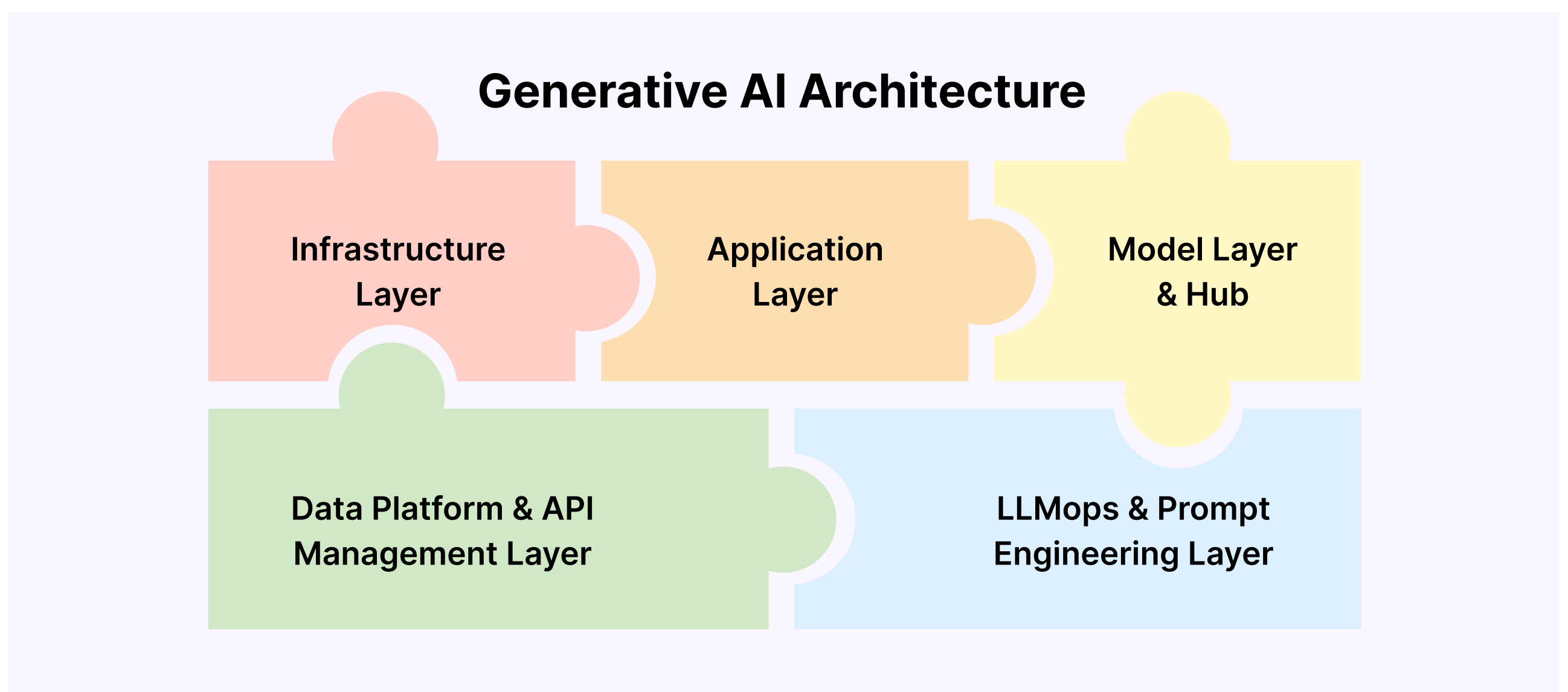
Example:

I am building a fintech app for Indian users.
Explain UPI transaction failures and solutions.

6. Generative AI Architecture

Modern Generative AI systems are built using layered architecture.

Each layer has a clearly defined responsibility, ensuring scalability, security, and real-world reliability.



6.1 Infrastructure Layer

The Infrastructure Layer provides the computational backbone required to run Generative AI models.

Generative models perform billions of matrix operations per request.

They require:

- High-performance hardware
- Parallel processing
- Intelligent scaling mechanisms

This layer ensures the system can handle real users at real scale.

| KEY RESPONSIBILITIES:

- Compute management (CPU / GPU / TPU)
- Auto-scaling based on traffic
- Fault tolerance and high availability
- Storage for models and logs
- Cost optimization

Example:

ChatGPT

- Runs on GPU clusters hosted on cloud infrastructure
- Automatically scales when millions of users send prompts
- Routes traffic through load balancers to avoid downtime

If infrastructure fails → the model still exists, but users get errors.

The Model Layer contains the core Generative AI models responsible for producing outputs.

Models learn patterns from data and generate predictions, but:

- They do not store knowledge like databases
- They do not understand context without input structure

A Model Hub manages multiple models for different tasks.

| KEY RESPONSIBILITIES:

- Hosting pre-trained models
- Managing fine-tuned models
- Switching models per use case
- Model versioning and A/B testing

Example:

E-commerce AI Assistant

- GPT- 4 → customer chat
- Smaller LLM → FAQ answering
- Vision model → product image analysis

The Model Hub decides which model serves which request.

This layer connects models with real-world data and systems.

Models cannot:

- Fetch data on their own
- Validate requests
- Manage structured pipelines

This layer acts as the control center for data and requests.

| KEY RESPONSIBILITIES:

- Data ingestion (databases, Kafka, warehouses)
- API routing and request validation
- Authentication & rate limiting
- Logging and monitoring requests
- Caching frequently used responses

Example:

Fintech Chatbot

(a) User asks: “Show my last transaction”

(b) API layer:

- Authenticates user
- Fetches data from bank database
- Sends relevant data to model
- Prevents model from seeing sensitive data directly

Without this layer → security violations happen.

6.4 LLMOps & Prompt Engineering Layer

LLMOps is the operational discipline for managing LLM behavior in production.

Prompt Engineering defines:

- How models are instructed
- How responses are constrained
- How errors are reduced

This layer ensures the model:

- Responds consistently
- Follows business rules
- Produces safe outputs

| KEY RESPONSIBILITIES:

- Prompt templates
- Output validation
- Guardrails and constraints

Example:

Customer Support AI

(a) Prompt includes:

- Company tone guidelines
- Legal disclaimers
- Escalation rules

(b) Output is validated to avoid hallucinations

Without prompt engineering → responses become unpredictable.

6.5

Application Layer

The Application Layer is what users actually interact with.

It turns model output into:

- Text chats
- Dashboards
- Images
- Recommendations

This layer handles UX, business logic, and orchestration.

| KEY RESPONSIBILITIES:

- UI and user flows
- Input preprocessing
- Output display

Example:

AI Resume Builder App

- User uploads resume
- Backend calls GenAI APIs
- UI displays rewritten resume
- User downloads PDF

The user never sees the model they see the application.

7. Future of Generative AI

Generative AI is moving very fast. In the future, it won't just write text or create images, but will work like a smart assistant that understands, thinks, and helps humans better.

Let's understand this step by step.

7.1 Trends And Innovations In Generative AI

What's changing?

Earlier, AI systems could do only one thing at a time:

- Either write text
- Or generate images
- Or answer basic questions

In the future, AI systems will become more smart, efficient, and practical.

1.1 Multimodal AI (One AI, Many Inputs)

| IN SIMPLE MEANING:

One AI model can understand text, images, videos, and audio together.

Example:

You upload:

- A picture of a bill
- Ask a question: *“How much did I spend on food this month?”*

The AI:

- Reads the image
- Understands the text inside it
- Answers your question

Just like humans use eyes + ears + brain together.

1.2 Smarter and Smaller Models

Earlier thinking was:

“Bigger AI model = Better AI”

Now the thinking is:

“Right-sized AI for the right job”

Why?

- Big models are expensive
- Slow to run
- Hard to control

Future AI:

- Smaller, faster AI models
- Trained for specific tasks (finance, healthcare, coding, etc.)

Benefit:

Cheaper, safer, and more accurate AI.

1.3 AI That Thinks Step by Step

Today's AI mostly predicts the next word.

Future AI will:

- Think in steps
- Explain its reasoning
- Plan before answering

Example:

Instead of just giving an answer, AI will say:

"First I checked this → then compared options → this is why this answer makes sense."

This makes AI more trustworthy and reliable.

In the future, companies won't use AI just for fun experiments.

They will use AI as:

- A work assistant
- A knowledge helper
- A decision support system

What AI will do at work

- Summarize long documents
- Help write reports
- Answer employee questions
- Analyze large amounts of data
- Help managers take better decisions

Important:

AI will support humans, not replace them.

Why companies are careful

Companies worry about:

- Data safety
- Wrong answers
- Legal issues
- Cost control

So future AI systems will have:

- Strict rules
- Human approvals
- Clear logs of what AI did

7.3

Human + AI Collaboration

This is the most important future concept.

AI is not replacing humans

AI is becoming a:

Smart assistant, not a boss

How humans and AI will work together

- Humans decide goals
- AI suggests options
- Humans make final decisions

Example:

AI suggests 5 business strategies

Human chooses the best one



WHY BOSSCODER?

01

STRUCTURED INDUSTRY-VETTED CURRICULUM

Our curriculum covers everything you need to get become a skilled software engineer & get placed.

02

1:1 MENTORSHIP SESSIONS

You are assigned a personal mentor currently working in Top product based companies.

03

2200+ ALUMNI PLACEMENT

2200+ Alumni placed at Top Product-based companies.

04

24 LPA AVERAGE PACKAGE

Our Average Placement Package is **24 LPA** and highest is **98 LPA**



Niranjan Bagade

Software Engineer,
British Petroleum

10 Years
Experience

NICE

Software Eng.
Specialist

Hike

83%



British Petroleum

Software Engineer



Dheeraj Barik

Software Engineer 2,
Amazon

2 Years
Experience

Infosys

Software Engineer

Hike

550%



Amazon

SDE 2

[EXPLORE MORE](#)